

SESSION 2 LAB PROGRAMS

C.Devasri 192521422

1. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

CODE:

```
#include <stdio.h>

int main() {
    int n, i, j, k = 0, found, a[100], u[100];
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d numbers: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < k; j++)
            if (u[j] == a[i]) {
                found = 1;
                break;
            }
        if (!found) u[k++] = a[i];
    }
    printf("Unique list: ");
    for (i = 0; i < k; i++) printf("%d ", u[i]);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
Enter n: 8
Enter 8 numbers: 3 7 3 5 2 5 9 2
Unique list: 3 7 5 2 9
```

```
Enter n: 6
Enter 6 numbers: -1 2 -1 3 2 -2
Unique list: -1 2 3 -2
```

```
Enter n: 3
Enter 3 numbers: 1000000 999999 1000000
Unique list: 1000000 999999
```

Space Complexity: $O(n)$. In the worst case (all elements are distinct) the new list `u[]` holds all `n` elements, so the extra space grows linearly with `n`.

Time Complexity: $O(n^2)$, because every element is compared with the unique elements found so far.

2.Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

CODE:

```
#include <stdio.h>

int main() {
    int n, i, j, t, swapped, a[100];
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d numbers: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
    printf("Sorted array: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
Enter n: 5
Enter 5 numbers: 64 34 25 12 22
Sorted array: 12 22 25 34 64
```

```
Enter n: 6
Enter 6 numbers: 5 1 4 2 8 0
Sorted array: 0 1 2 4 5 8
```

Time Complexity Analysis:

Worst case (array in reverse order): $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ comparisons and swaps, so $O(n^2)$.

Average case: $O(n^2)$.

Best case (already sorted): with the swapped flag the loop stops after one pass, so $O(n)$.

Space Complexity: $O(1)$, because the array is sorted in place.

3. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

CODE:

```
#include <stdio.h>

int main() {
    int n, i, x, low, high, mid, pos = -1, arr[100];
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d sorted numbers: ", n);
    for (i = 0; i < n; i++) scanf("%d", &arr[i]);
    printf("Enter x: ");
    scanf("%d", &x);

    low = 0;
    high = n - 1;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == x) {
            pos = mid + 1;
            break;
        } else if (arr[mid] < x)
            low = mid + 1;
        else
            high = mid - 1;
    }

    if (pos != -1)
        printf("Element %d is found at position %d\n", x, pos);
    else
        printf("Element %d is not found\n", x);
    return 0;
}
```

OUTPUT:

```
Enter n: 8
Enter 8 sorted numbers: -9 3 4 6 8 9 10 30
Enter x: 10
Element 10 is found at position 7
```

```
Enter n: 8
Enter 8 sorted numbers: -9 3 4 6 8 9 10 30
Enter x: 100
Element 100 is not found
```

Time Complexity Analysis:

Each step of binary search halves the search range: n , $n/2$, $n/4$, ... 1. The number of halvings is $\log_2 n$, so the search takes $O(\log n)$ in the worst case and $O(1)$ in the best case (key at the middle). Space Complexity: $O(1)$.

4. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

CODE:

```
#include <stdio.h>

void siftDown(int a[], int i, int n) {
    int largest, l, r, t;
    while (1) {
        largest = i;
        l = 2 * i + 1;
        r = 2 * i + 2;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest == i) break;
        t = a[i];
        a[i] = a[largest];
        a[largest] = t;
        i = largest;
    }
}

int main() {
    int n, i, t, a[100];
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Enter %d numbers: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    /* build max heap */
    for (i = n / 2 - 1; i >= 0; i--) siftDown(a, i, n);

    /* repeatedly move the max to the end */
    for (i = n - 1; i > 0; i--) {
        t = a[0];
        a[0] = a[i];
        a[i] = t;
    }
}
```

```
        siftDown(a, 0, i);
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
Enter n: 4
Enter 4 numbers: 5 2 3 1
Sorted array: 1 2 3 5
```

```
Enter n: 6
Enter 6 numbers: 5 1 1 2 0 0
Sorted array: 0 0 1 1 2 5
```

Heap sort is used: Time Complexity $O(n \log n)$ in best, average and worst case, and Space Complexity $O(1)$, since the array is sorted in place and siftDown is written iteratively (no recursion stack).